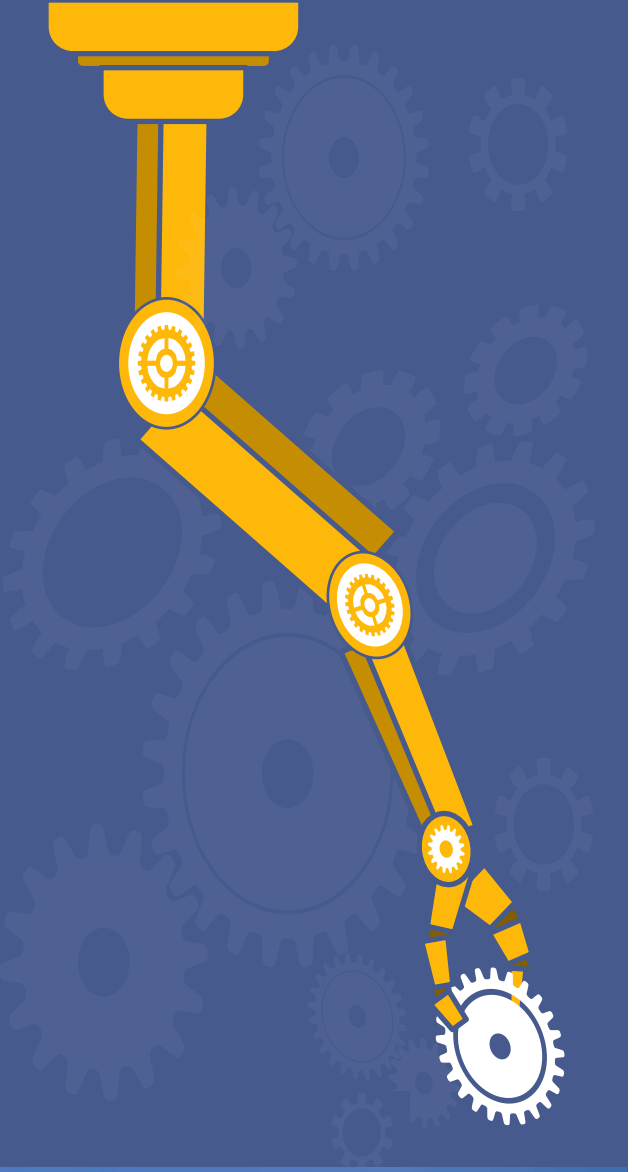


VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT, Vellore.

BCSE204L- Design and Analysis of Algorithms-

Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms

2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively

3. To synthesize efficient algorithms in various engineering design situations

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms

2. Demonstrate the major algorithm design paradigms.

3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.

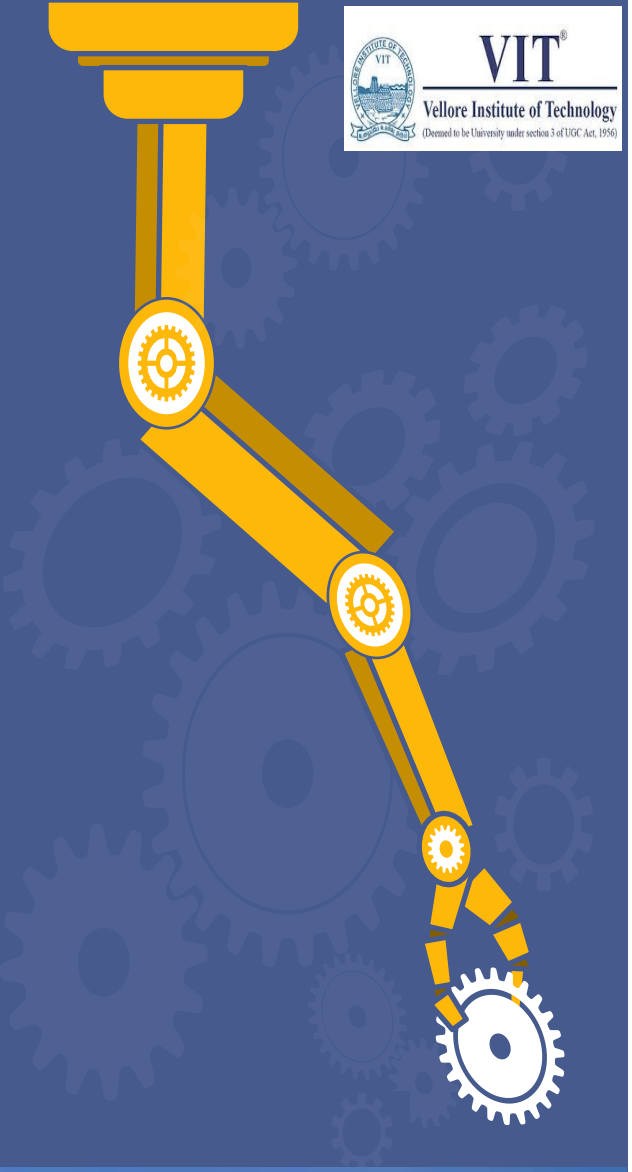
4. Articulating Randomized Algorithms.

5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	**Design Paradigms: Greedy, Divide and Conquer Techniques**	**6 hours**
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	**Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques**	**10 hours**
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	**String Matching Algorithms**	**5 hours**
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	**Graph Algorithms**	**6 hours**
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	**Geometric Algorithms**	**4 hours**
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	**Randomized algorithms**	**5 hours**
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	**Classes of Complexity and Approximation Algorithms**	**7 hours**
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	**Contemporary Issues**	**2 hours**



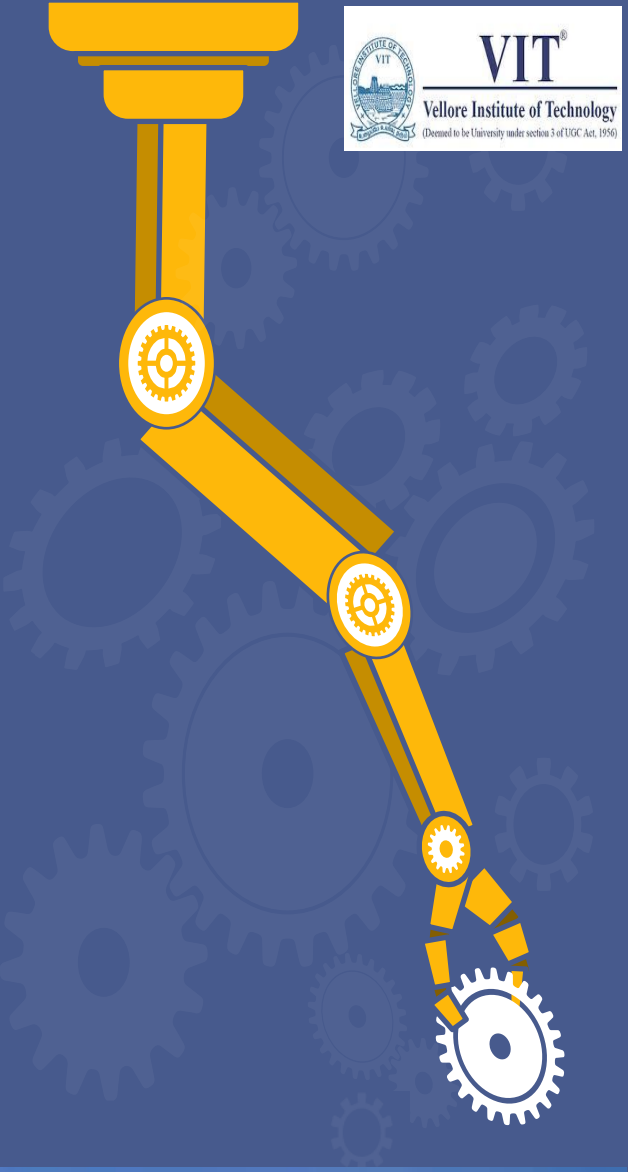
BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, ÉvaTardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press,1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B.Orlin, " Network Flows: Theory, Algorithms and Applications", Pearson Education, 2014.



Module 2: Design Paradigms: Dynamic Programming-Backtracking- Branch and Bound(10 Hours)

- **Dynamic Programming**
  - Matrix chain Multiplication
  - Assembly line Scheduling
  - Longest Common Subsequence
  - **0/1 Knapsack Problem**
  - TSP- Travelling Salesman Problem
- **Backtracking**
  - N-queens Problem
  - Sum of Subsets
  - Graph Coloring
- **Branch and Bound**
  - LIFO and FIFO Methods
  - Job Selection Problem
  - 0/1 Knapsack Problem

Dynamic Programming

Dynamic programming is an algorithm design method that can be used when the solution to a problem can be viewed as the result of a sequence of decisions.

Principle of Optimality:

The principle of optimality states that an optimal sequence of decisions has the property that whatever the initial state and decision are, the remaining decisions must constitute an optimal decision sequence with regard to the state resulting from the first decision.

0/1 Knapsack Problem - Idea

We are given n objects and a knapsack. Object **i** has a weight $\mathbf{w_i}$ and the knapsack has a capacity **M.** **Objects may not be broken into smaller pieces. So We decide that either to take an object or leave it behind, but fraction of objects should not be taken. So x_i will be 0 if not taken else 1 if we include the object.** The objective is to obtain a filling of the knapsack that maximizes the total profit earned. Since the knapsack capacity is **M,** we require the total weight of all chosen objects to be at most **M**.

0/1 Knapsack Problem - Idea

Formally the Problem can be Stated as:

$$\text{maximize} \sum_{1 \le i \le n} p_i x_i \quad \longrightarrow \quad (1)$$

$$\text{subject to} \sum_{1 \le i \le n} w_i x_i \le m \quad \longrightarrow \quad (2)$$

The profits $\mathbf{p_i}$ and weights $\mathbf{w_i}$ are positive numbers.

$x_i \in \{0, 1\}$

Knapsack Problem

0-1 version

Knapsack problem.

- Given n objects and a "knapsack."
- Item i weighs $w_i > 0$ kilograms and has value $v_i > 0$.
- Knapsack has capacity of W kilograms.
- Goal: fill knapsack so as to maximize total value.

Ex: { 3, 4 } has value 40.

W = 11

Item	Value	Weight	v/w
1	1	1	1,
2	6	2	3,
3	18	5	3.6,
4	22	6	3.66,
5	28	7	4.

Greedy: repeatedly add item with maximum ratio v_i / w_i.

Ex: { 5, 2, 1 } achieves only value = 35 ⇒ greedy not optimal.

- To Solve the problem using Dynamic programming set up a table name $V[W_1:W_N, V_0:V_N]$
  - **Create one row for each available object**
  - **Create one column for each weight starting from 0 to w.**

		0	1	2	3	4	5	6	7	8	9	10	11
1	W1=1,V1=1	0	1	1	1	1	1	1	1	1	1	1	1
2	W2=2,V2=6	0	1	6	7	7	7	7	7	7	7	7	7
3	W3=5,V3=18	0	1	6	7	7	18	19	24	25	25	25	25
4	W4=6,V4=22	0	1	6	7	7	18	22	24	28	29	29	40
5	W5=7,V5=28	0	1	6	7	7	18	22	28	29	34	35	40

- **Formula for filling the table**

$$V[i,j] = \text{Max}[v[i-1,j],\ v[i-1,j-w_i] + v_i]$$

Not adding object i to the load

Choosing object **i** which has effect to increase the value of load by $\mathbf{v_i}$ and to reduce the capacity available by $\mathbf{w_i}$

Knapsack Problem: Bottom-Up

Knapsack. Fill up an n-by-W array.

```
Input: n, w1,…,wN, v1,…,vN

for w = 0 to W
   M[0, w] = 0

for i = 1 to n
   for w = 1 to W
      if (wi > w)
         M[i, w] = M[i-1, w]
      else
         M[i, w] = max {M[i-1, w], vi + M[i-1, w-wi ]}

return M[n, W]
```

Knapsack Algorithm

W + 1 →

n + 1 ↓

	0	1	2	3	4	5	6	7	8	9	10	11
ϕ	0	0	0	0	0	0	0	0	0	0	0	0
{ 1 }	0	1	1	1	1	1	1	1	1	1	1	1
{ 1, 2 }	0	1	6	7	7	7	7	7	7	7	7	7
{ 1, 2, 3 }	0	1	6	7	7	18	19	24	25	25	25	25
{ 1, 2, 3, 4 }	0	1	6	7	7	18	22	24	28	29	29	40
{ 1, 2, 3, 4, 5 }	0	1	6	7	7	18	22	28	29	34	34	40

35

Item	Value	Weight
1	1	1
2	6	2
3	18	5
4	22	6
5	28	7

W = 11

OPT: { 4, 3 }
value = 22 + 18 = 40

- Rules to find the optimal load
  - If v[i,j]= v[i-1,j], then the object is not included
  - If $v[i,j]= v[i-1,j-w_i] + v_i$, Then the object is included

- The Solution to the problem starts from v[n,w]
  - **V[5,11]= v[4,11], 40=40 then the object 5 is not included**
  - V[4,11] ⇏ v[3,11], 40 ⇏ 25, check for next rule
  - V[4,11] = v[3,5]+22, 40 = 40 object 4 is included
  - V[3,5]= v[2,5], 18 ⇏ 7, check for next rule.
  - V[3,5]= V[2,0]+18, 18= 18 object 3 is included.
  - **V[2,0]= V[1,0], 0=0, then the object 2 is not included**
  - **V[1,0]= V[0,0], 0=0, then the object 1 is not included**

Conclusion

- In this problem there is only one optimal load consisting of objects 3 and 4.
- In case of greedy, we have to consider object 5 since it has greatest value per unit weight, Next we have to consider object 4, but its not included, By continuing this, it looks on objects 3,2,1, and **ends with object 5,2,1 which has profit value as 35.** so Greedy does not work when objects cannot be broken.
- Time for constructing the table: $O(nW)$
- Time taken for Composition of Optimal load: $O(n+W)$

Thank You